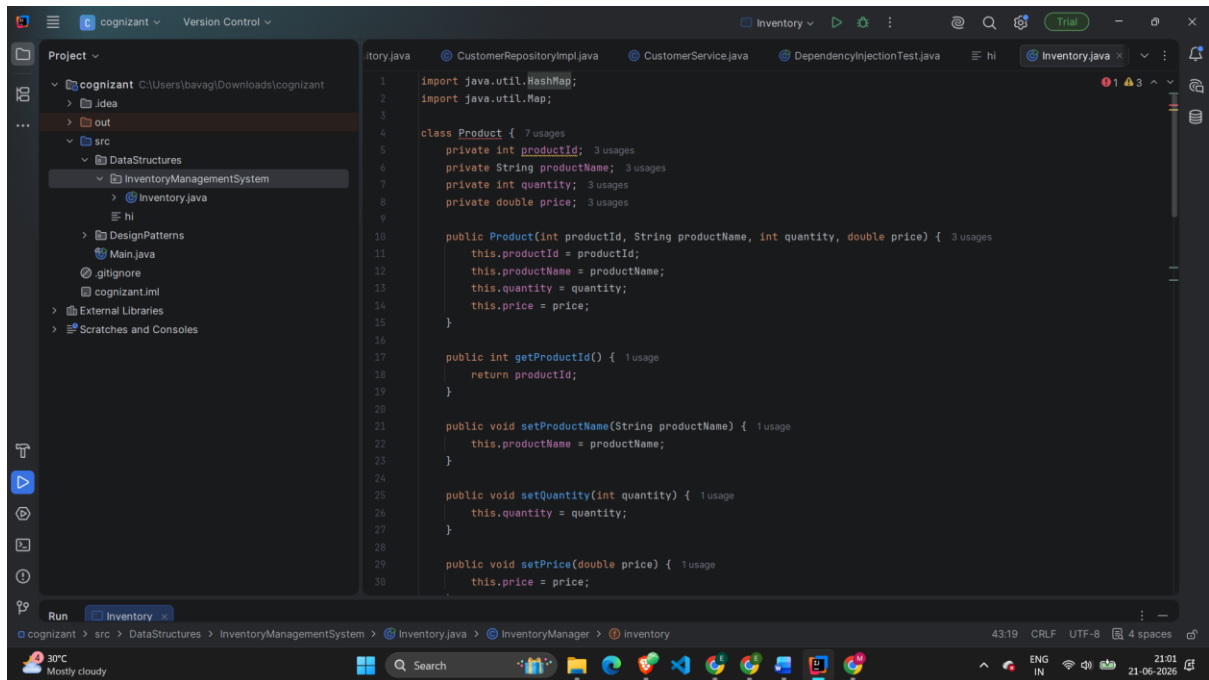


## Module-2 {DataStructures}

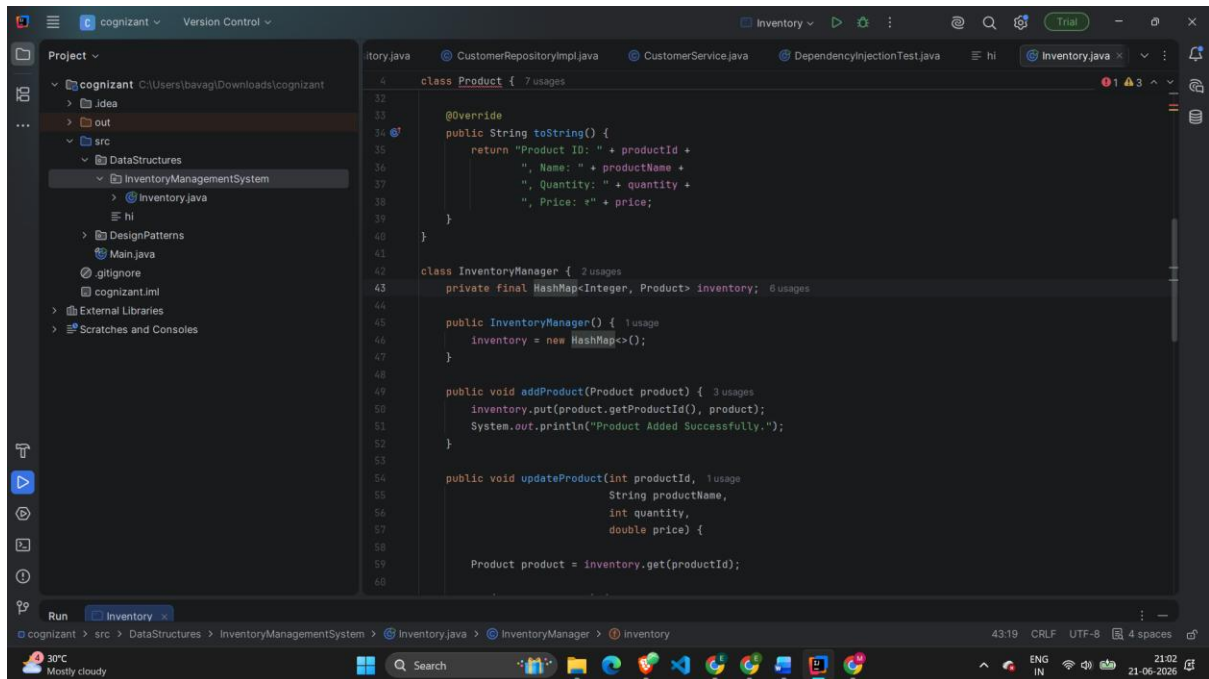
### Exercise 1: Inventory Management System



This screenshot shows the IntelliJ IDEA interface with the `Inventory.java` file open. The `Product` class is defined with the following attributes and methods:

```
1 import java.util.HashMap;
2 import java.util.Map;
3
4 class Product {
5     private int productId;
6     private String productName;
7     private int quantity;
8     private double price;
9
10    public Product(int productId, String productName, int quantity, double price) {
11        this.productId = productId;
12        this.productName = productName;
13        this.quantity = quantity;
14        this.price = price;
15    }
16
17    public int getProductId() {
18        return productId;
19    }
20
21    public void setProductName(String productName) {
22        this.productName = productName;
23    }
24
25    public void setQuantity(int quantity) {
26        this.quantity = quantity;
27    }
28
29    public void setPrice(double price) {
30        this.price = price;
31    }
32 }
```

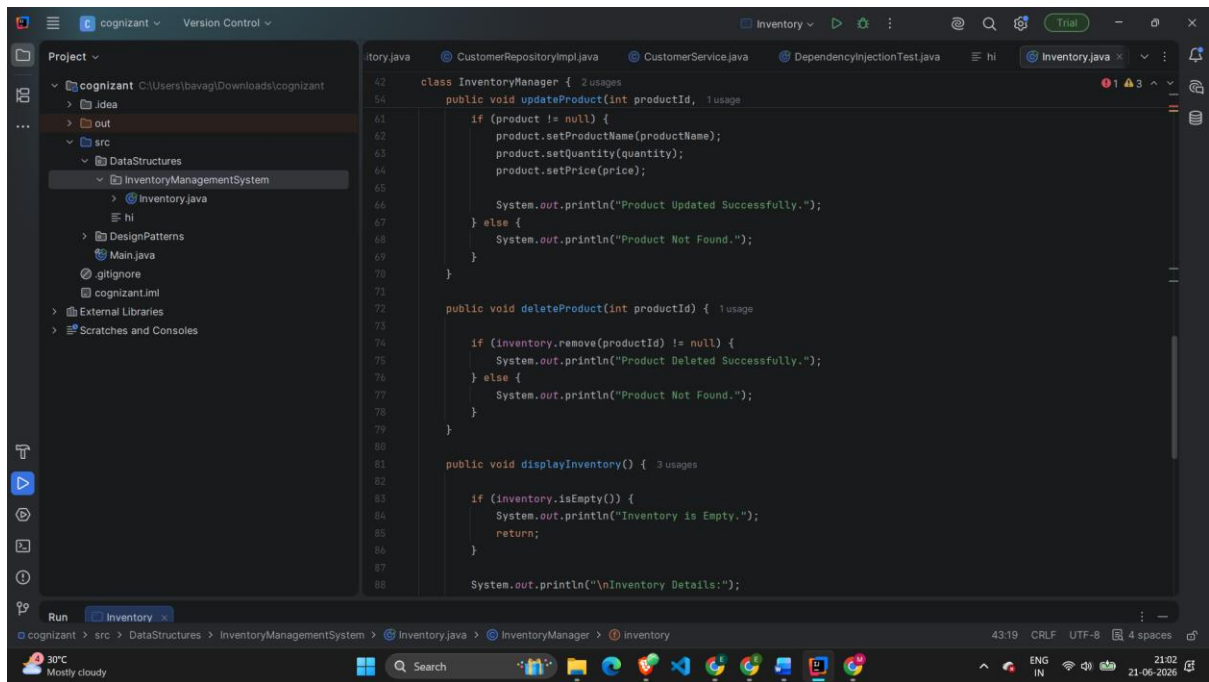
The project structure on the left shows the file is located at `src > DataStructures > InventoryManagementSystem > Inventory.java`. The bottom status bar indicates the file is in `UTF-8` encoding with 4 spaces.



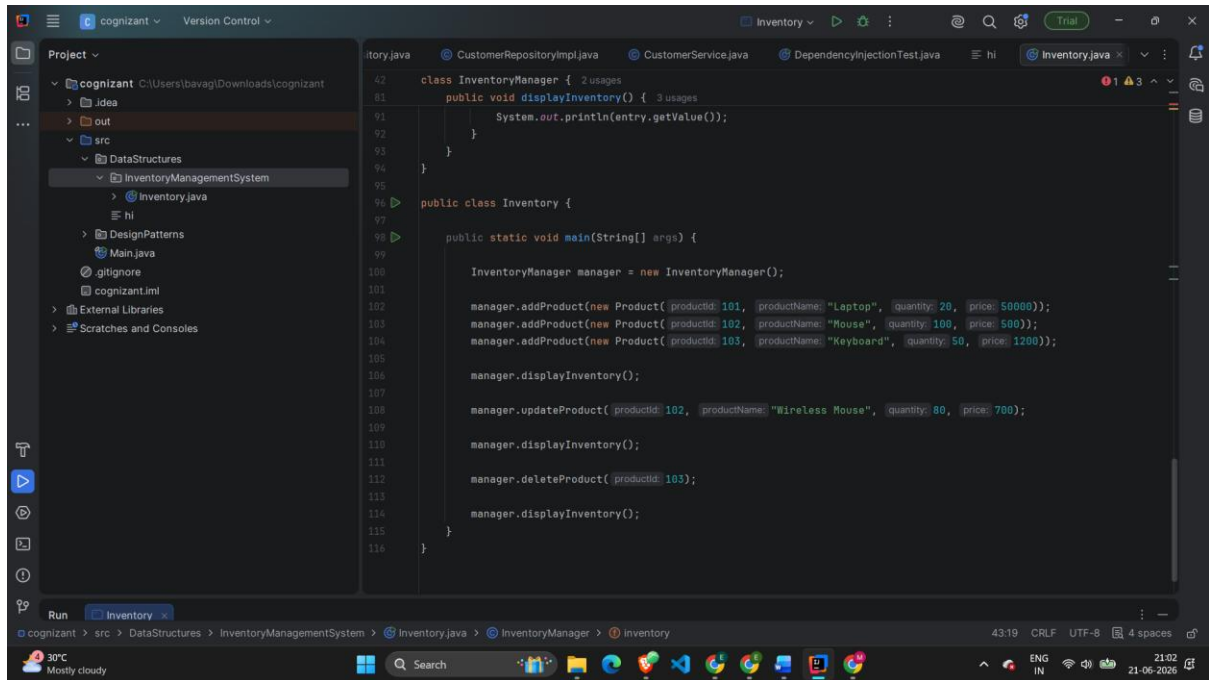
This screenshot shows the `InventoryManager` class in the same `Inventory.java` file. It includes an `toString` method for the `Product` class and the `InventoryManager` class logic:

```
32
33
34 @Override
35 public String toString() {
36     return "Product ID: " + productId +
37         ", Name: " + productName +
38         ", Quantity: " + quantity +
39         ", Price: " + price;
40 }
41
42 class InventoryManager {
43     private final HashMap<Integer, Product> inventory;
44
45     public InventoryManager() {
46         inventory = new HashMap<>();
47     }
48
49     public void addProduct(Product product) {
50         inventory.put(product.getProductId(), product);
51         System.out.println("Product Added Successfully.");
52     }
53
54     public void updateProduct(int productId,
55                             String productName,
56                             int quantity,
57                             double price) {
58
59         Product product = inventory.get(productId);
60     }
61 }
```

The project structure remains the same. The bottom status bar shows the file is in `UTF-8` encoding with 4 spaces.

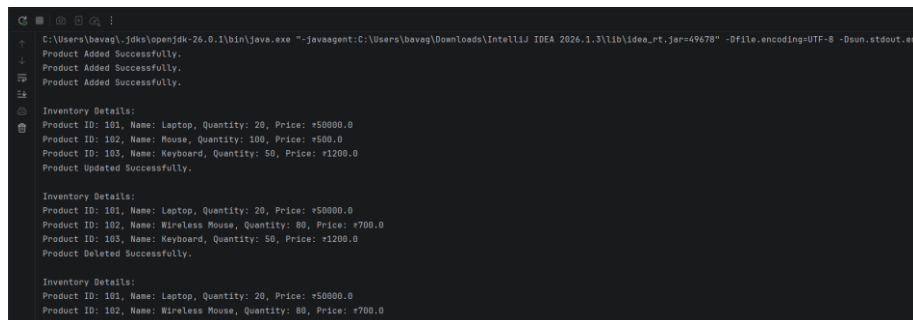


```
42 class InventoryManager { 2 usages
43     public void updateProduct(int productId, 1 usage
44     {
45         if (product != null) {
46             product.setProductName(productName);
47             product.setQuantity(quantity);
48             product.setPrice(price);
49
50             System.out.println("Product Updated Successfully.");
51         } else {
52             System.out.println("Product Not Found.");
53         }
54     }
55
56     public void deleteProduct(int productId) { 1 usage
57     {
58         if (inventory.remove(productId) != null) {
59             System.out.println("Product Deleted Successfully.");
60         } else {
61             System.out.println("Product Not Found.");
62         }
63     }
64
65     public void displayInventory() { 3 usages
66     {
67         if (inventory.isEmpty()) {
68             System.out.println("Inventory is Empty.");
69             return;
70         }
71
72         System.out.println("\nInventory Details:");
73     }
74 }
```



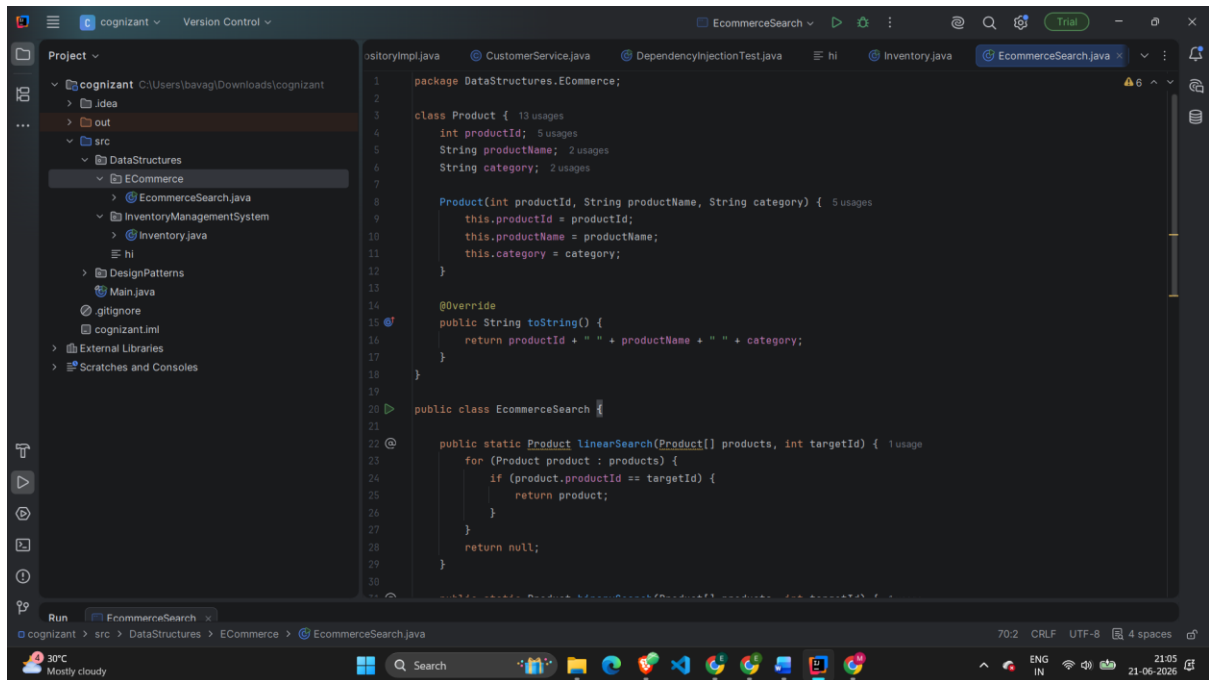
```
91     System.out.println(entry.getValue());
92 }
93 }
94 }
95
96 public class Inventory {
97
98     public static void main(String[] args) {
99
100         InventoryManager manager = new InventoryManager();
101
102         manager.addProduct(new Product(101, "Laptop", 20, 50000));
103         manager.addProduct(new Product(102, "Mouse", 100, 500));
104         manager.addProduct(new Product(103, "Keyboard", 50, 1200));
105
106         manager.displayInventory();
107
108         manager.updateProduct(102, "Wireless Mouse", 80, 700);
109
110         manager.displayInventory();
111
112         manager.deleteProduct(103);
113
114         manager.displayInventory();
115     }
116 }
```

Output:



```
C:\Users\bavag\jdkopenjdk-26.0.1\bin\java.exe -javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar-49678" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
Product Added Successfully.
Product Added Successfully.
Product Added Successfully.
Inventory Details:
Product ID: 101, Name: Laptop, Quantity: 20, Price: ₹50000.0
Product ID: 102, Name: Mouse, Quantity: 100, Price: ₹500.0
Product ID: 103, Name: Keyboard, Quantity: 50, Price: ₹1200.0
Product Updated Successfully.
Inventory Details:
Product ID: 101, Name: Laptop, Quantity: 20, Price: ₹50000.0
Product ID: 102, Name: Wireless Mouse, Quantity: 80, Price: ₹700.0
Product ID: 103, Name: Keyboard, Quantity: 50, Price: ₹1200.0
Product Deleted Successfully.
Inventory Details:
Product ID: 101, Name: Laptop, Quantity: 20, Price: ₹50000.0
Product ID: 102, Name: Wireless Mouse, Quantity: 80, Price: ₹700.0
```

## Exercise 2: E-commerce Platform Search Function



The screenshot shows the IntelliJ IDEA IDE with the project 'cognizant' open. The file explorer on the left shows the project structure, including 'DataStructures' and 'ECommerce'. The main editor displays the 'EcommerceSearch.java' file. The code defines a 'Product' class with attributes 'productId', 'productName', and 'category'. It also implements a 'linearSearch' method that iterates through an array of products to find a specific product by its ID.

```
package DataStructures.ECommerce;

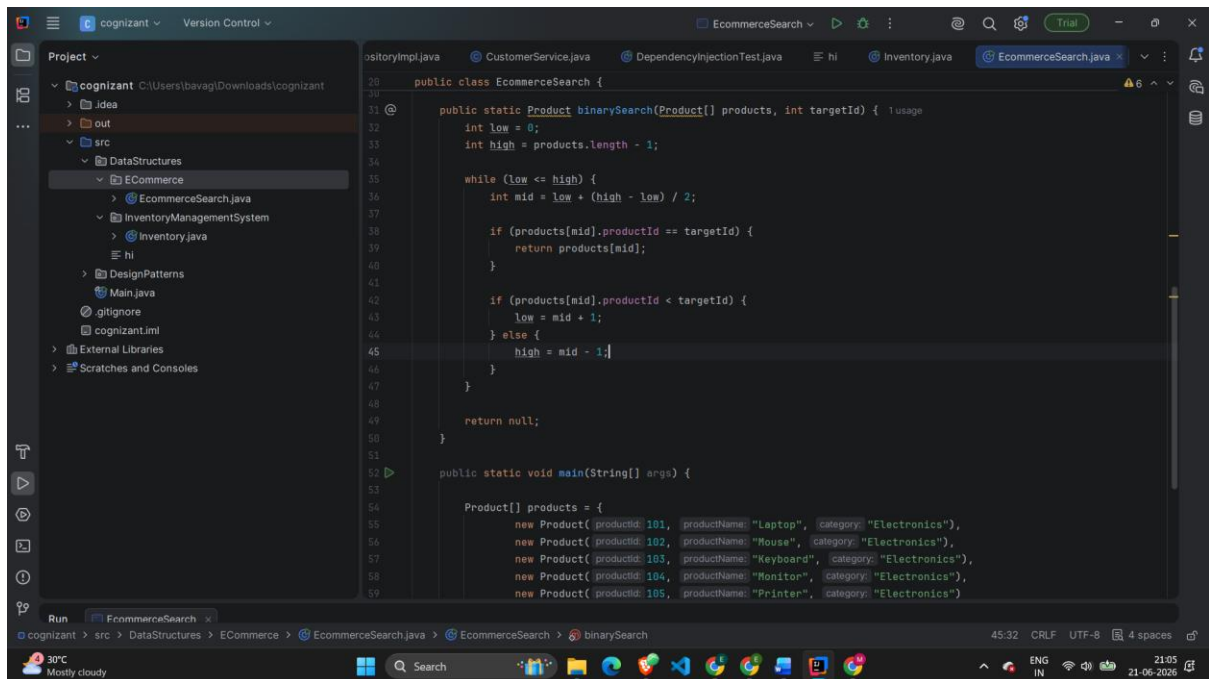
class Product {
    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return productId + " " + productName + " " + category;
    }
}

public class EcommerceSearch {

    public static Product linearSearch(Product[] products, int targetId) {
        for (Product product : products) {
            if (product.productId == targetId) {
                return product;
            }
        }
        return null;
    }
}
```



The screenshot shows the IntelliJ IDEA IDE with the project 'cognizant' open. The file explorer on the left shows the project structure, including 'DataStructures' and 'ECommerce'. The main editor displays the 'EcommerceSearch.java' file. The code defines a 'Product' class with attributes 'productId', 'productName', and 'category'. It also implements a 'binarySearch' method that uses a binary search algorithm to find a specific product by its ID. Additionally, a 'main' method is provided to test the search function with a sample array of products.

```
public class EcommerceSearch {

    public static Product binarySearch(Product[] products, int targetId) {
        int low = 0;
        int high = products.length - 1;

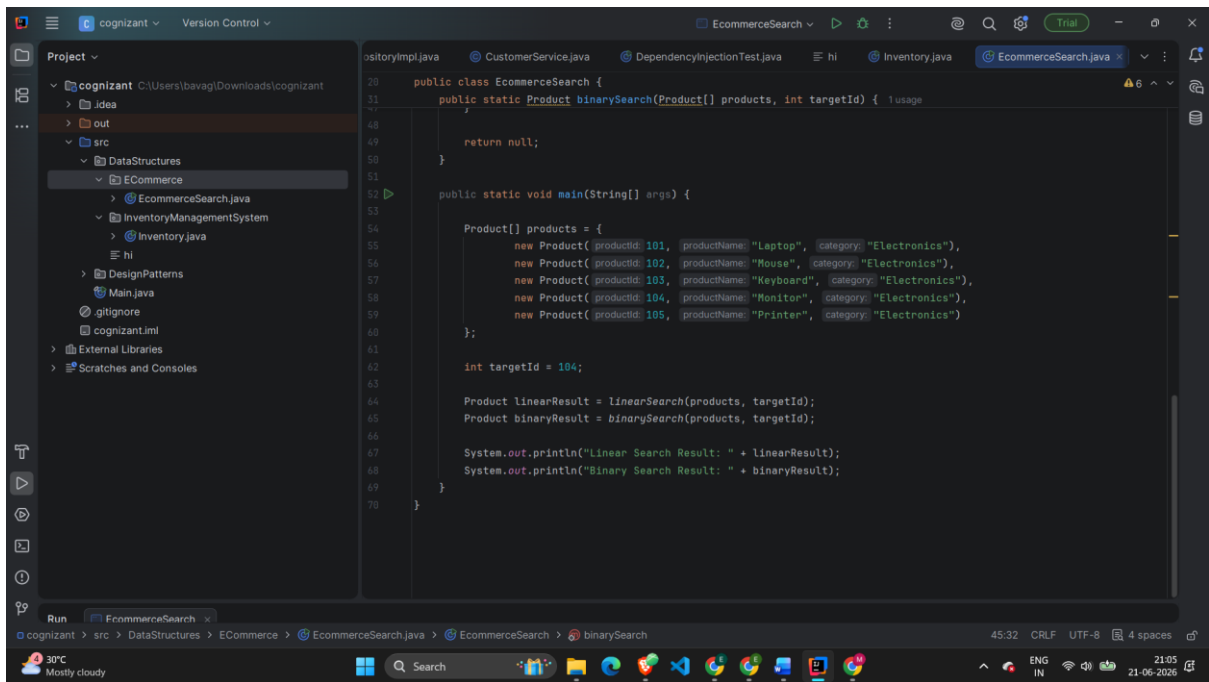
        while (low <= high) {
            int mid = (low + high) / 2;

            if (products[mid].productId == targetId) {
                return products[mid];
            }

            if (products[mid].productId < targetId) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

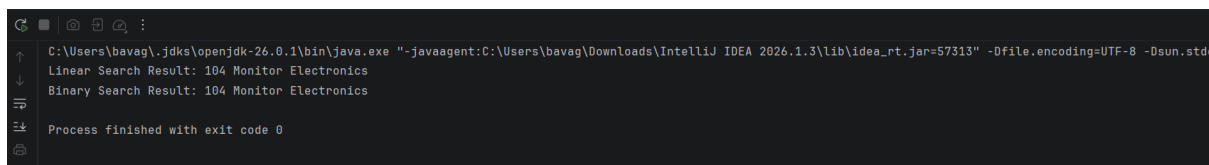
        return null;
    }

    public static void main(String[] args) {
        Product[] products = {
            new Product(101, "Laptop", "Electronics"),
            new Product(102, "Mouse", "Electronics"),
            new Product(103, "Keyboard", "Electronics"),
            new Product(104, "Monitor", "Electronics"),
            new Product(105, "Printer", "Electronics")
        };
    }
}
```



```
28 public class EcommerceSearch {
29     public static Product binarySearch(Product[] products, int targetId) { 1 usage
30     }
31
32     return null;
33 }
34
35 public static void main(String[] args) {
36
37     Product[] products = {
38         new Product( productid: 101, productName: "Laptop", category: "Electronics"),
39         new Product( productid: 102, productName: "Mouse", category: "Electronics"),
40         new Product( productid: 103, productName: "Keyboard", category: "Electronics"),
41         new Product( productid: 104, productName: "Monitor", category: "Electronics"),
42         new Product( productid: 105, productName: "Printer", category: "Electronics")
43     };
44
45     int targetId = 104;
46
47     Product linearResult = linearSearch(products, targetId);
48     Product binaryResult = binarySearch(products, targetId);
49
50     System.out.println("Linear Search Result: " + linearResult);
51     System.out.println("Binary Search Result: " + binaryResult);
52 }
53 }
```

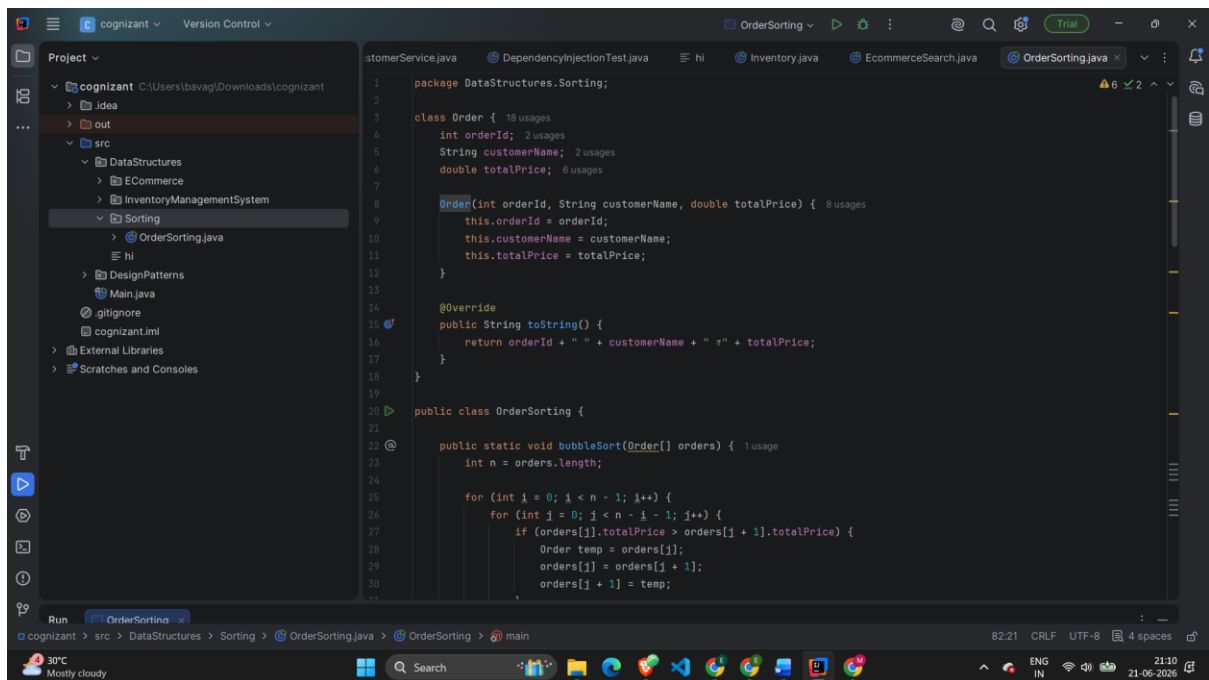
Output:



```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=57513" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
Linear Search Result: 104 Monitor Electronics
Binary Search Result: 104 Monitor Electronics

Process finished with exit code 0
```

## Exercise 3: Sorting Customer Orders



```
1 package DataStructures.Sorting;
2
3 class Order { 18 usages
4     int orderId; 2 usages
5     String customerName; 2 usages
6     double totalPrice; 6 usages
7
8     Order(int orderId, String customerName, double totalPrice) { 8 usages
9         this.orderId = orderId;
10        this.customerName = customerName;
11        this.totalPrice = totalPrice;
12    }
13
14    @Override
15    public String toString() {
16        return orderId + " " + customerName + " " + totalPrice;
17    }
18 }
19
20 public class OrderSorting {
21
22     public static void bubbleSort(Order[] orders) { 1 usage
23         int n = orders.length;
24
25         for (int i = 0; i < n - 1; i++) {
26             for (int j = 0; j < n - i - 1; j++) {
27                 if (orders[j].totalPrice > orders[j + 1].totalPrice) {
28                     Order temp = orders[j];
29                     orders[j] = orders[j + 1];
30                     orders[j + 1] = temp;
31                 }
32             }
33         }
34     }
35 }
```

The screenshot shows the IntelliJ IDEA IDE with the 'OrderSorting.java' file open. The project structure on the left includes 'cognizant' > 'src' > 'DataStructures' > 'Sorting'. The code in the editor defines a 'public class OrderSorting' with two static methods: 'bubbleSort' and 'quickSort'. The 'bubbleSort' method takes an 'Order[] orders' array and sorts it using a bubble sort algorithm. The 'quickSort' method takes an 'Order[] orders' array and 'int low, int high' indices, and sorts it using a quick sort algorithm. The 'partition' method is also defined as a static method. The bottom status bar shows the file is at 'cognizant > src > DataStructures > Sorting > OrderSorting.java' and the cursor is at line 62, column 1.

```
20 public class OrderSorting {
21     public static void bubbleSort(Order[] orders) { 1 usage
22         for (int i = 0; i < orders.length - 1; i++) {
23             for (int j = i + 1; j < orders.length; j++) {
24                 if (orders[i].totalPrice > orders[j].totalPrice) {
25                     Order temp = orders[i];
26                     orders[i] = orders[j];
27                     orders[j] = temp;
28                 }
29             }
30         }
31     }
32
33     public static void quickSort(Order[] orders, int low, int high) { 3 usages
34         if (low < high) {
35             int pivotIndex = partition(orders, low, high);
36             quickSort(orders, low, pivotIndex - 1);
37             quickSort(orders, pivotIndex + 1, high);
38         }
39     }
40
41     public static int partition(Order[] orders, int low, int high) { 1 usage
42         double pivot = orders[high].totalPrice;
43         int i = low - 1;
44
45         for (int j = low; j < high; j++) {
46             if (orders[j].totalPrice <= pivot) {
47                 i++;
48                 Order temp = orders[i];
49                 orders[i] = orders[j];
50                 orders[j] = temp;
51             }
52         }
53
54         Order temp = orders[i + 1];
55         orders[i + 1] = orders[high];
56         orders[high] = temp;
57         return i + 1;
58     }
59 }
```

The screenshot shows the IntelliJ IDEA IDE with the 'OrderSorting.java' file open. The code in the editor defines a 'public class OrderSorting' with two static methods: 'printOrders' and 'main'. The 'printOrders' method takes an 'Order[] orders' array and prints each order's details. The 'main' method takes a 'String[] args' array and creates two arrays of 'Order' objects, 'orders1' and 'orders2', and sorts them using the 'bubbleSort' and 'quickSort' methods respectively. The 'main' method also prints the sorted orders. The bottom status bar shows the file is at 'cognizant > src > DataStructures > Sorting > OrderSorting.java' and the cursor is at line 85, column 1.

```
44 public static int partition(Order[] orders, int low, int high) { 1 usage
45     Order temp = orders[high];
46     orders[high] = orders[low];
47     orders[low] = temp;
48
49     return low + 1;
50 }
51
52 public static void printOrders(Order[] orders) { 2 usages
53     for (Order order : orders) {
54         System.out.println(order);
55     }
56     System.out.println();
57 }
58
59 public static void main(String[] args) {
60     Order[] orders1 = {
61         new Order(101, "Praneeth", 5000),
62         new Order(102, "Rahul", 2500),
63         new Order(103, "Kiran", 8000),
64         new Order(104, "Sai", 3500)
65     };
66
67     Order[] orders2 = {
68         new Order(101, "Praneeth", 5000),
69         new Order(102, "Rahul", 2500),
70         new Order(103, "Kiran", 8000),
71         new Order(104, "Sai", 3500)
72     };
73
74     bubbleSort(orders1);
75     printOrders(orders1);
76
77     System.out.println("Quick Sort:");
78     quickSort(orders2, 0, orders2.length - 1);
79     printOrders(orders2);
80 }
```

The screenshot shows the IntelliJ IDEA IDE with the 'OrderSorting.java' file open. The code in the editor defines a 'public class OrderSorting' with a 'main' method. The 'main' method takes a 'String[] args' array and creates two arrays of 'Order' objects, 'orders1' and 'orders2', and sorts them using the 'bubbleSort' and 'quickSort' methods respectively. The 'main' method also prints the sorted orders. The bottom status bar shows the file is at 'cognizant > src > DataStructures > Sorting > OrderSorting.java' and the cursor is at line 95, column 1.

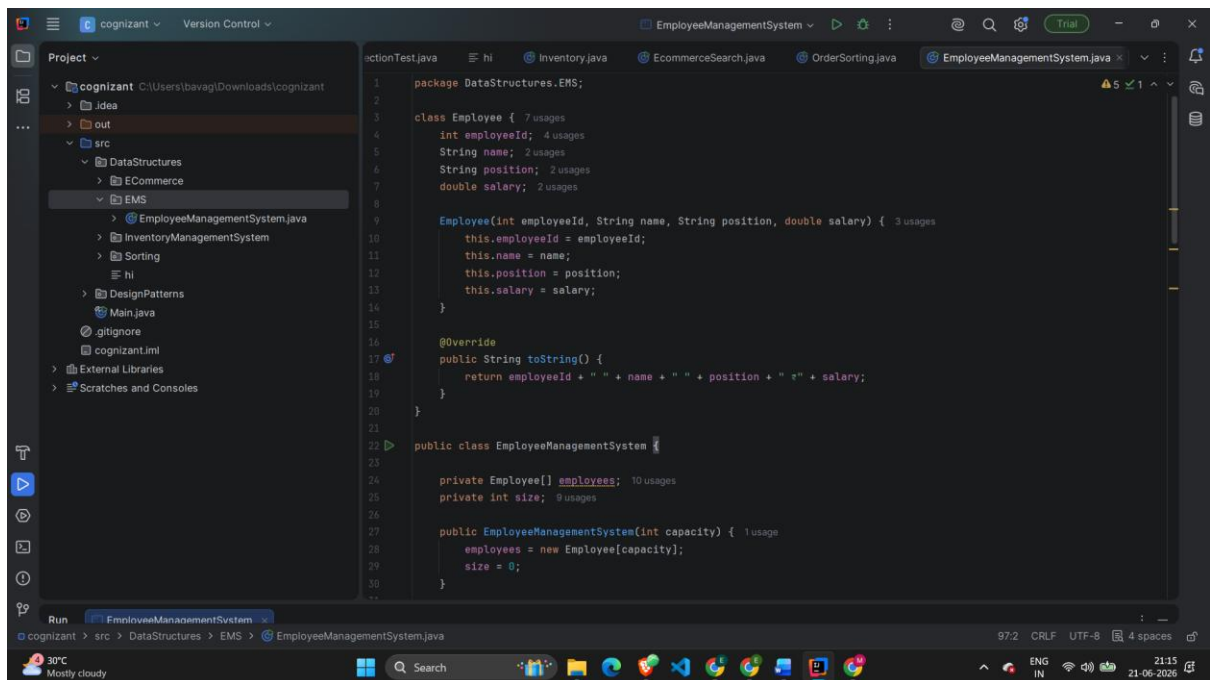
```
85     System.out.println("Bubble Sort:");
86     bubbleSort(orders1);
87     printOrders(orders1);
88
89     System.out.println("Quick Sort:");
90     quickSort(orders2, 0, orders2.length - 1);
91     printOrders(orders2);
92 }
```

Output:

```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=51381" -Dfile.encoding=UTF-8 -Dsun.s
Bubble Sort:
102 Rahul ₹2500.0
104 Sai ₹3500.0
101 Praneeth ₹5000.0
103 Kiran ₹8000.0

Quick Sort:
102 Rahul ₹2500.0
104 Sai ₹3500.0
101 Praneeth ₹5000.0
103 Kiran ₹8000.0
```

## Exercise 4: Employee Management System





```
22 public class EmployeeManagementSystem {
23
24     public void addEmployee(Employee employee) { 3 usages
25         if (size < employees.length) {
26             employees[size++] = employee;
27         } else {
28             System.out.println("Array is full");
29         }
30     }
31
32     public Employee searchEmployee(int employeeId) { 1 usage
33         for (int i = 0; i < size; i++) {
34             if (employees[i].employeeId == employeeId) {
35                 return employees[i];
36             }
37         }
38         return null;
39     }
40
41     public void traverseEmployees() { 2 usages
42         for (int i = 0; i < size; i++) {
43             System.out.println(employees[i]);
44         }
45     }
46
47     public void deleteEmployee(int employeeId) { 1 usage
48         int index = -1;
49
50         for (int i = 0; i < size; i++) {
51             if (employees[i].employeeId == employeeId) {
```

```
55     public void deleteEmployee(int employeeId) { 1 usage
56
57         index = i;
58         break;
59     }
60
61     if (index == -1) {
62         System.out.println("Employee not found");
63         return;
64     }
65
66     for (int i = index; i < size - 1; i++) {
67         employees[i] = employees[i + 1];
68     }
69
70     employees[size - 1] = null;
71     size--;
72 }
73
74 public static void main(String[] args) {
75
76     EmployeeManagementSystem ems = new EmployeeManagementSystem(10);
77
78     ems.addEmployee(new Employee(101, "Praneeth", "Developer", 50000));
79     ems.addEmployee(new Employee(102, "Rahul", "Tester", 40000));
80     ems.addEmployee(new Employee(103, "Kiran", "Manager", 70000));
81
82     System.out.println("Employees:");
83     ems.traverseEmployees();
```

```
88
89     System.out.println("\nSearch Employee:");
90     System.out.println(ems.searchEmployee(employeeId: 102));
91
92     ems.deleteEmployee(employeeId: 102);
93
94     System.out.println("\nAfter Deletion:");
95     ems.traverseEmployees();
96 }
97 }
```

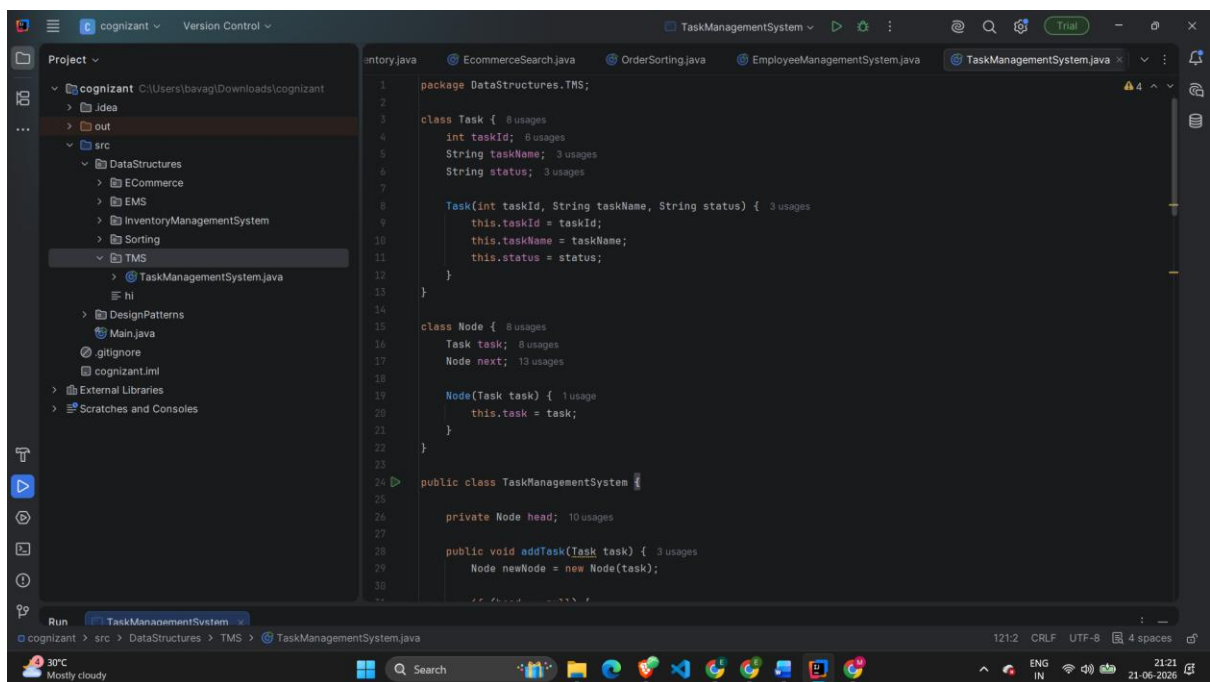
Output:

```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=57454" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8
Employees:
101 Praneeth Developer ₹50000.0
102 Rahul Tester ₹40000.0
103 Kiran Manager ₹70000.0

Search Employee:
102 Rahul Tester ₹40000.0

After Deletion:
101 Praneeth Developer ₹50000.0
103 Kiran Manager ₹70000.0
```

## Exercise 5: Task Management System



```
package DataStructures.TMS;

class Task {
    int taskID;
    String taskName;
    String status;

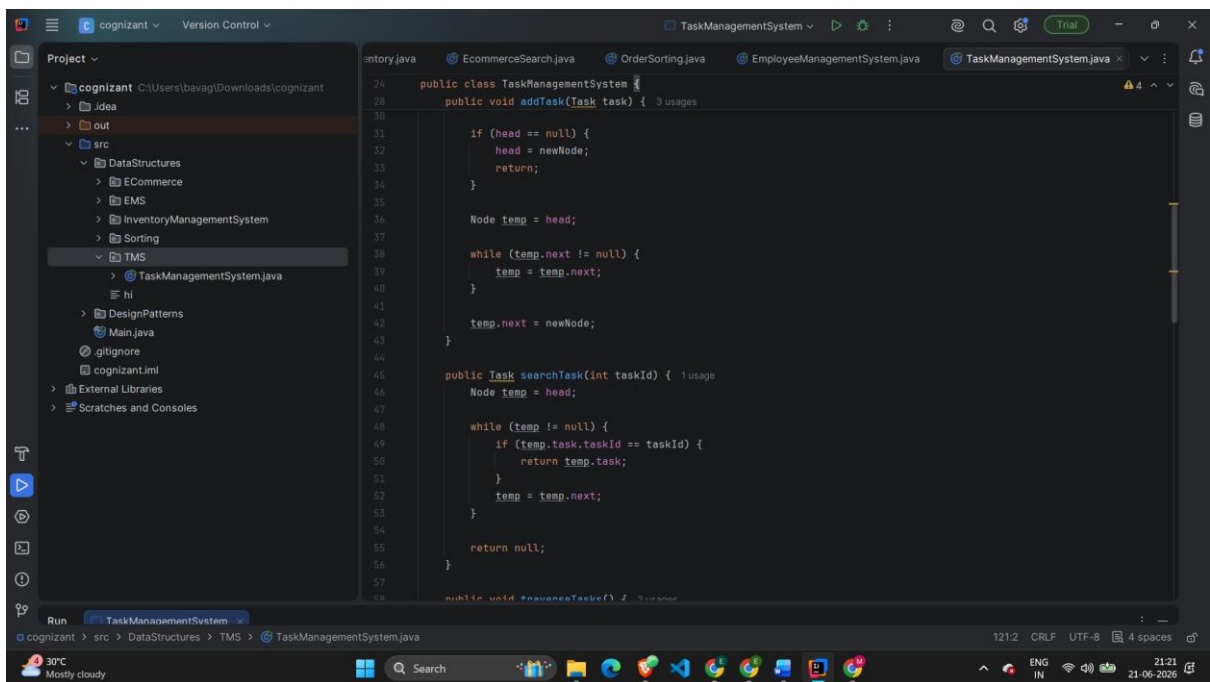
    Task(int taskID, String taskName, String status) {
        this.taskID = taskID;
        this.taskName = taskName;
        this.status = status;
    }
}

class Node {
    Task task;
    Node next;

    Node(Task task) {
        this.task = task;
    }
}

public class TaskManagementSystem {
    private Node head;

    public void addTask(Task task) {
        Node newNode = new Node(task);
    }
}
```



```
public class TaskManagementSystem {
    private Node head;

    public void addTask(Task task) {
        if (head == null) {
            head = new Node(task);
            return;
        }

        Node temp = head;
        while (temp.next != null) {
            temp = temp.next;
        }
        temp.next = new Node(task);
    }

    public Task searchTask(int taskID) {
        Node temp = head;
        while (temp != null) {
            if (temp.task.taskID == taskID) {
                return temp.task;
            }
            temp = temp.next;
        }
        return null;
    }
}
```



```
24 public class TaskManagementSystem {
57
58     public void traverseTasks() { 2 usages
59         Node temp = head;
60
61         while (temp != null) {
62             System.out.println(
63                 temp.task.taskId + " " +
64                 temp.task.taskName + " " +
65                 temp.task.status
66             );
67             temp = temp.next;
68         }
69     }
70
71     public void deleteTask(int taskId) { 1 usage
72
73         if (head == null) {
74             return;
75         }
76
77         if (head.task.taskId == taskId) {
78             head = head.next;
79             return;
80         }
81
82         Node temp = head;
83
84         while (temp.next != null &&
85             temp.next.task.taskId != taskId) {
```

```
74 public class TaskManagementSystem {
75     public void deleteTask(int taskId) { 1 usage
76         temp.next.task.taskId = taskId;
77         temp = temp.next;
78     }
79
80     if (temp.next == null) {
81         temp.next = temp.next.next;
82     }
83 }
84
85 public static void main(String[] args) {
86     TaskManagementSystem tasks = new TaskManagementSystem();
87
88     tasks.addTask(new Task(taskId: 101, taskName: "Design UI", status: "Pending"));
89     tasks.addTask(new Task(taskId: 102, taskName: "Develop Backend", status: "In Progress"));
90     tasks.addTask(new Task(taskId: 103, taskName: "Testing", status: "Pending"));
91
92     System.out.println("Tasks:");
93     tasks.traverseTasks();
94
95     System.out.println("\nSearch Task:");
96     Task found = tasks.searchTask(taskId: 102);
97
98     if (found != null) {
99         System.out.println(
100             found.taskId + " " +
101             found.taskName + " " +
102             found.status
103         );
104     }
105 }
106
107 }
```

```
112         found.status
113     );
114 }
115
116 tasks.deleteTask(taskId: 102);
117
118 System.out.println("\nAfter Deletion:");
119 tasks.traverseTasks();
120 }
121 }
```

Output:

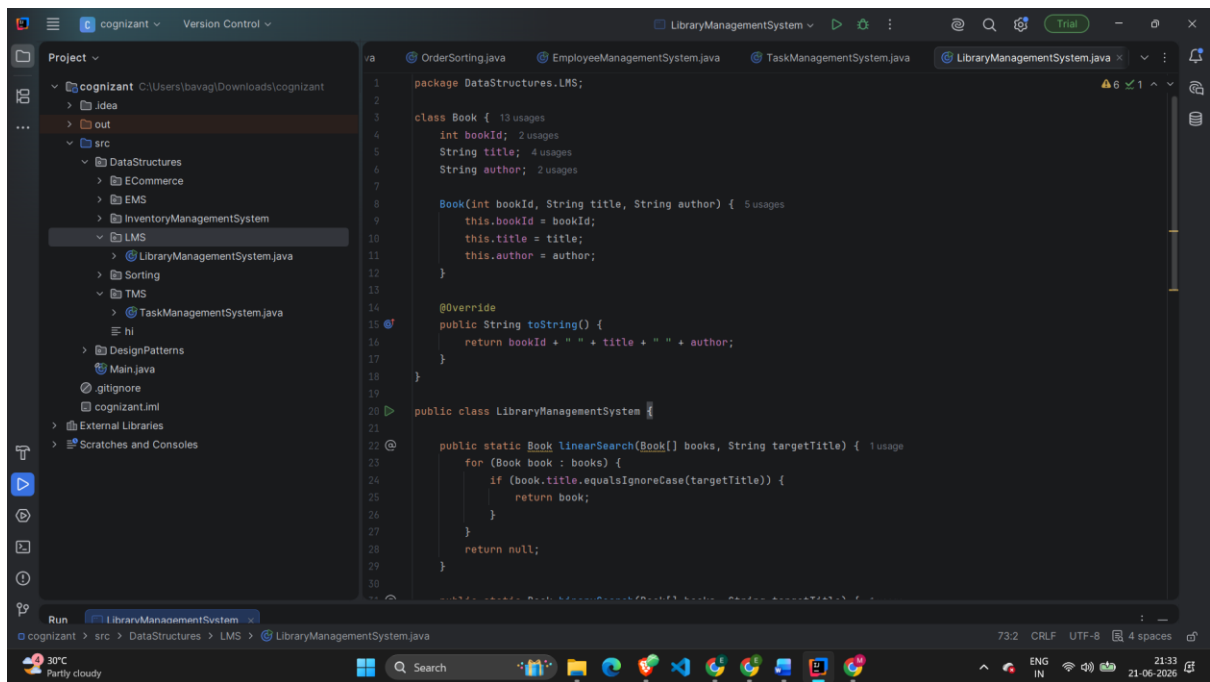
```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=62352" -Dfile.encoding=UTF-8

Tasks:
101 Design UI Pending
102 Develop Backend In Progress
103 Testing Pending

Search Task:
102 Develop Backend In Progress

After Deletion:
101 Design UI Pending
103 Testing Pending
```

## Exercise 6: Library Management System



```
package DataStructures.LMS;

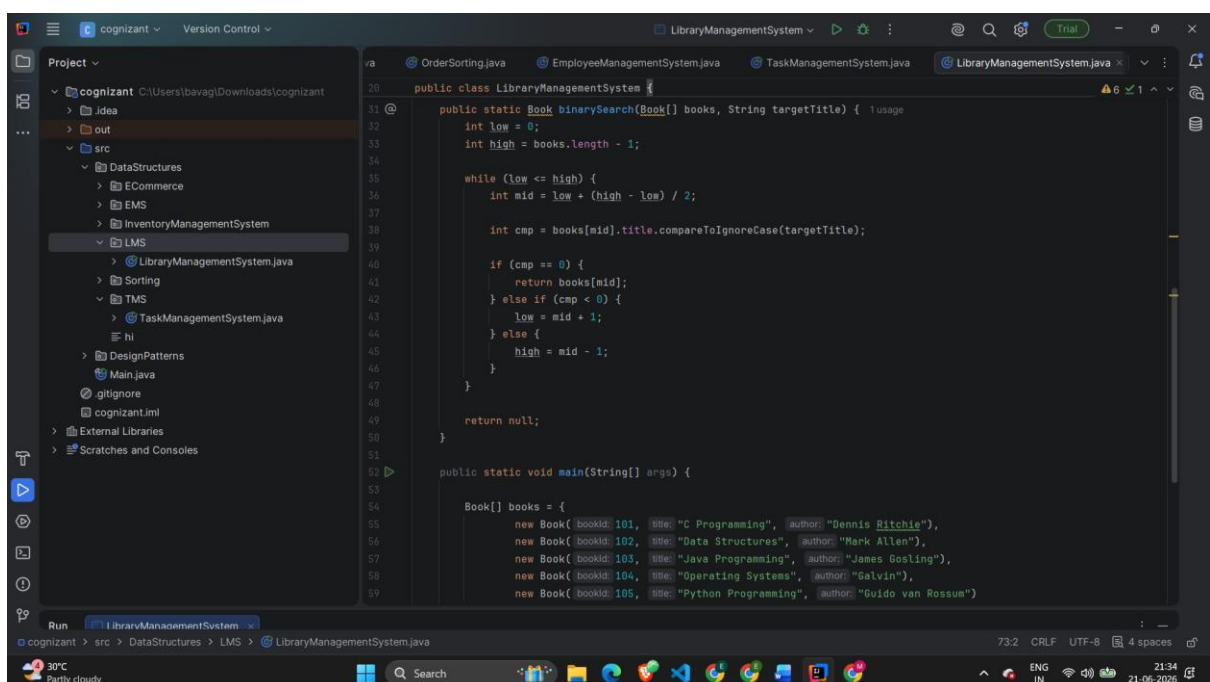
class Book {
    int bookId;
    String title;
    String author;

    Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    @Override
    public String toString() {
        return bookId + " " + title + " " + author;
    }
}

public class LibraryManagementSystem {

    public static Book linearSearch(Book[] books, String targetTitle) {
        for (Book book : books) {
            if (book.title.equalsIgnoreCase(targetTitle)) {
                return book;
            }
        }
        return null;
    }
}
```



```
public class LibraryManagementSystem {

    public static Book binarySearch(Book[] books, String targetTitle) {
        int low = 0;
        int high = books.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            int cmp = books[mid].title.compareToIgnoreCase(targetTitle);

            if (cmp == 0) {
                return books[mid];
            } else if (cmp < 0) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        return null;
    }

    public static void main(String[] args) {

        Book[] books = {
            new Book(101, "C Programming", "Dennis Ritchie"),
            new Book(102, "Data Structures", "Mark Allen"),
            new Book(103, "Java Programming", "James Gosling"),
            new Book(104, "Operating Systems", "Galvin"),
            new Book(105, "Python Programming", "Guido van Rossum")
        };
    }
}
```

```
        new Book( bookId: 104, title: "Operating Systems", author: "Galvin"),
        new Book( bookId: 105, title: "Python Programming", author: "Guido van Rossum")
    };

    String targetTitle = "Java Programming";

    Book linearResult = linearSearch(books, targetTitle);
    Book binaryResult = binarySearch(books, targetTitle);

    System.out.println("Linear Search Result:");
    System.out.println(linearResult);

    System.out.println("\nBinary Search Result:");
    System.out.println(binaryResult);
}
```

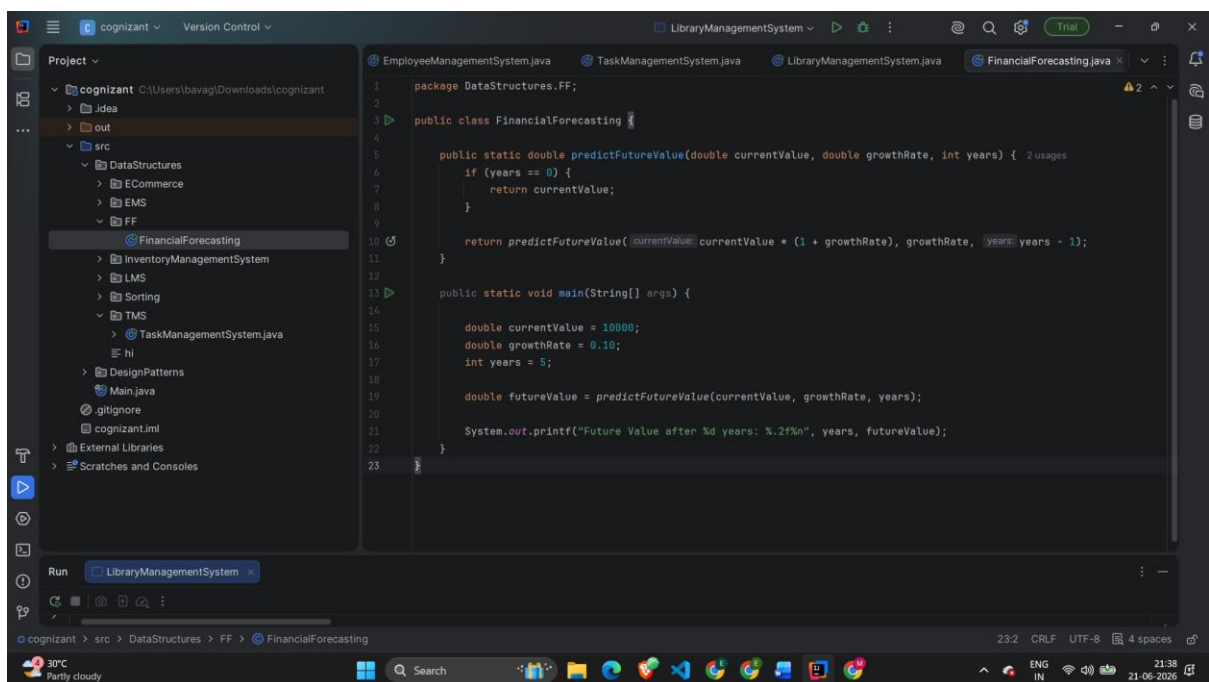
Output:

```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=58478" -Dfile.encoding=UTF-8 -Dsu
Linear Search Result:
103 Java Programming James Gosling

Binary Search Result:
103 Java Programming James Gosling

Process finished with exit code 0
```

## Exercise 7: Financial Forecasting



The screenshot shows the IntelliJ IDEA IDE interface. On the left, the Project Explorer displays a project named 'cognizant' with a source folder 'src' containing several packages: 'DataStructures', 'ECommerce', 'EMS', 'FF', 'InventoryManagementSystem', 'LMS', 'Sorting', 'TMS', 'TaskManagementSystem.java', 'hi', 'DesignPatterns', and 'Main.java'. The 'FF' package is expanded, showing 'FinancialForecasting'. The main editor displays the 'FinancialForecasting.java' file with the following code:

```
1 package DataStructures.FF;
2
3 public class FinancialForecasting {
4
5     public static double predictFutureValue(double currentValue, double growthRate, int years) {
6         if (years == 0) {
7             return currentValue;
8         }
9
10        return predictFutureValue(currentValue * (1 + growthRate), growthRate, years - 1);
11    }
12
13    public static void main(String[] args) {
14
15        double currentValue = 10000;
16        double growthRate = 0.10;
17        int years = 5;
18
19        double futureValue = predictFutureValue(currentValue, growthRate, years);
20
21        System.out.printf("Future Value after %d years: %.2f\n", years, futureValue);
22    }
23 }
```

The bottom status bar shows the file path 'cognizant > src > DataStructures > FF > FinancialForecasting', the encoding 'UTF-8', and the line length '23:2'.

## Output:

A screenshot of a terminal window with a dark background. The terminal shows a Java command being executed, which outputs the future value after 5 years. The command is partially visible and includes a file path for the IntelliJ IDEA runtime jar. The output is a single line of text. Below the output, a message indicates the process finished successfully.

```
C:\Users\bavag\.jdk\openjdk-26.0.1\bin\java.exe "-javaagent:C:\Users\bavag\Downloads\IntelliJ IDEA 2026.1.3\lib\idea_rt.jar=57218" -Dfile.encoding=UTF-8
Future Value after 5 years: 16105.10
Process finished with exit code 0
```